

BigBird

Sparse attention for long sequences

This literature review examines why BigBird was developed, how its architecture and block-sparse attention work, and how it compares with BERT and Longformer. It also covers the model's innovations, applications, advantages, limitations, and published performance. No model training or runtime benchmarking was carried out for this review.

1. Why BigBird is needed

In full self-attention, every token is compared with every other token. An input of n tokens therefore produces n^2 attention scores per head. BERT uses bidirectional Transformer encoders and was trained with sequences of up to 512 tokens. Its paper identifies the quadratic cost of attention as a difficulty when working with longer sequences. [3]

Input tokens	Full-attention scores per head
1,000	1,000,000
2,000	4,000,000
4,096	16,777,216

Doubling the input length quadruples the number of attention scores. The figures above are calculated score counts; they do not represent parameter counts or measured runtimes. BigBird addresses this growth by retaining selected attention connections, which makes much longer context practical. [5]

2. Architecture

BigBird keeps the main components of a Transformer: embeddings, positional information, attention heads, feed-forward layers, residual connections, and normalization. Its main architectural change is the pattern of attention connections. The official repository includes BERT/RoBERTa-like encoder models as well as Pegasus encoder-decoder variants. [6]

The commonly used google/bigbird-roberta-base checkpoint uses masked-language-model pretraining and accepts up to 4,096 input tokens. This is the limit of that checkpoint, so its context is still finite. [7]

3. The three attention patterns

BigBird combines local connections, global tokens, and random links. Each permitted connection receives a learned attention weight. Randomness determines which positions are connected; it does not make their attention weights random. [5]

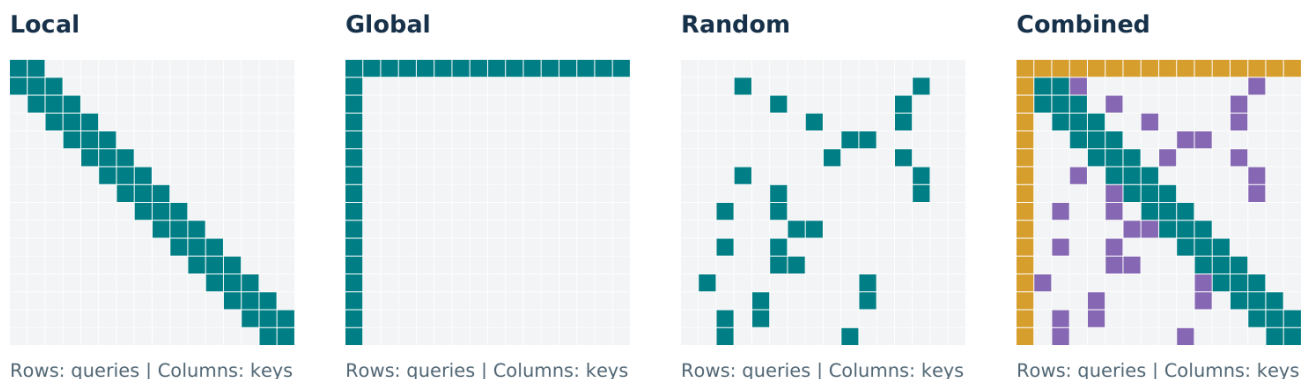


Figure 1. Illustrative token masks for local, global, random, and combined attention. Rows represent queries and columns represent keys. The masks show connectivity, rather than measured weights or the exact pattern of a particular checkpoint.

Local attention

With local, or sliding-window, attention, each token attends to a limited neighborhood. Nearby words often provide useful grammatical and contextual information. The diagonal band in Figure 1 shows these local connections. [5]

Global attention

A small set of global tokens attends to the whole sequence, and every token attends to those global tokens. This lets them act as communication hubs. In the ITC variant, existing input tokens are made global; the ETC variant introduces extra global tokens. [1]

Random attention

Each token also attends to a small number of randomly selected positions outside its local neighborhood. These links provide additional routes for information to travel between distant parts of a document. [5]

How the patterns work together

Local links preserve nearby context, global links distribute document-wide information, and random links add distant connections. Combining all three gives BigBird more routes for exchanging information than a local window alone. [5]

For example, a project name might appear near the beginning of a CV, while its evaluation metrics appear near the end. Local attention helps interpret each passage, and the additional connections allow information to pass between them. This example illustrates the mechanism; it is not a measured result.

The paper also establishes theoretical expressivity results, including universal approximation and Turing completeness, for suitable sparse Transformers under stated assumptions. These results do not guarantee equal accuracy at a fixed depth or in every application. [1]

4. Block sparsity and computational cost

From token connections to blocks

A block-sparse implementation groups consecutive tokens into blocks. For an allowed pair of blocks, it calculates a dense tile of token-to-token attention scores. Disallowed block pairs are skipped. Organizing the sparse connections this way allows the work to be performed through matrix operations suited to accelerators. [1,6]

Consider an input of 4,096 tokens divided into blocks of 64. This produces 64 blocks, and each allowed block pair contains $64 \times 64 = 4,096$ attention scores per head. The implementation calculates selected tiles instead of every tile in the full grid.

Hugging Face documents a local window of three blocks and two global blocks. The default block size is 64, with three random blocks per query block. Input lengths must be compatible with the block size, and the documented implementation supports ITC. [4]

Comparison with BERT and Longformer

Model	Attention pattern	Attention scaling
BERT	Full bidirectional attention. [3]	$O(n^2)$
Longformer	Local window and task-selected global tokens; the paper also explores dilation. [2]	$O(n(w + g))$
BigBird	Local, global, and random connections. [1,5]	$O(n(w + r + g))$

Here, n is the sequence length, w is the local-neighborhood size, r is the number of random connections, and g is the number of global tokens. These expressions omit feature dimensions and constant factors. When w , r , and g remain fixed, both Longformer and BigBird scale as $O(n)$. If g increases with n , that linear scaling can be lost.

At the block level, the work for selected scores is proportional to $n \times b \times k$, where b is the block size and k is the number of attended blocks. With fixed b and k , this work grows linearly; global blocks add further linear work. This describes the scaling, rather than giving an exact score count for a checkpoint.

Longformer already provides linear attention. BigBird's distinction is its addition of random links and the theoretical analysis associated with that pattern. Sparse attention changes the connections being computed; the number of model parameters does not fall by the same ratio. [1,2]

5. Applications and published performance

Applications

Published evaluations cover long-context question answering, document classification, summarization, and genomics. Longer context is useful when the evidence needed for a task is spread across a sequence. Summarization uses an encoder-decoder variant, while an encoder-only checkpoint requires a suitable task head. [1,4,6]

An example from the original paper

Table 4 of the BigBird paper reports the following arXiv summarization results for large models. Higher ROUGE scores indicate greater overlap with reference summaries. These are the authors' published results; they were not produced as part of this assignment. [1]

Large model	ROUGE-1	ROUGE-2	ROUGE-L
Pegasus (re-evaluated)	43.85	16.83	39.17
BigBird-Pegasus	46.63	19.02	41.77

The comparison supports BigBird in the tested long-document setting. It does not show that BigBird consistently outperforms BERT, Longformer, or other models. The paper's performance claims refer to its 2020 experiments, rather than the current leaderboard. [1]

Advantages

Common BigBird checkpoints support inputs of 4,096 tokens. Selected attention connections avoid the growth of a full quadratic score matrix, and pretrained encoder and summarization variants are available for reuse. These features make the model useful for tasks that need more context. [4,6,7]

Limitations and practical performance

For short inputs, the overhead of sparse attention can outweigh its benefits. Hugging Face recommends full attention below 1,024 tokens. Checkpoints still have finite context limits, and actual runtime depends on padding, block size, kernels, hardware, and batch size. [4,7]

Some token pairs have no direct connection, so information must pass through intermediate states. The paper shows that certain problems require more layers when sparse attention is used. Longer context also does not, by itself, guarantee better predictions. [1]

Relevance to the internship

BigBird would be worth evaluating for long CVs or project descriptions. Its extra context capacity offers a less obvious benefit for the earlier short-query classifier. This is an inference about suitability, rather than an experimental finding. Whether to use retrieval-augmented generation (RAG) remains a separate design decision.

References

- [1] Zaheer et al. (2020). Big Bird: Transformers for Longer Sequences. NeurIPS. Sections 2-5, Table 4, Appendix D. [Source](#).
- [2] Beltagy, Peters & Cohan (2020). Longformer: The Long-Document Transformer. Section 3. [Source](#).
- [3] Devlin et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL; first posted in 2018. [Source](#).
- [4] Hugging Face. Transformers BigBird documentation. Implementation notes and configuration. [Source](#).
- [5] Dubey (2021). Constructing Transformers For Longer Sequences with Sparse Attention Methods. Google Research. [Source](#).
- [6] Google Research. Official BigBird repository. Model variants and implementation. [Source](#).
- [7] Google/Hugging Face. bigbird-roberta-base model card. Card authored by Hugging Face. [Source](#).

Sources checked on 7 October 2026. The diagrams and arithmetic examples are explanatory illustrations.